

Software Engineering in the Era of AI

Revised course outline for 2026 to 2027

First year master's students in Electrical and Information Engineering. Sixteen weeks, three contact hours per week, comprising two hours of lecture and one hour of laboratory or assessed project activity. The nominal semester allocation is 48 contact hours. Week 16 uses the two lecture hours for the final examination and the laboratory hour for project presentations and individual defence.

This course develops the ability to specify, implement, evaluate and maintain software when AI can generate code and act through development tools. Students learn Python through a small engineering application, then investigate what evidence is needed to trust both AI-assisted development and applications that contain AI. The central responsibility is to explain and defend engineering decisions, including when a deterministic solution is preferable.

The revised syllabus retains the seven-module structure and the original assessment weights. It strengthens programming support, requirements engineering, reproducibility, evaluation and security. It introduces coding agents, context engineering, structured model outputs, retrieval-augmented generation, tool protocols and operational monitoring. Advanced infrastructure is treated selectively so that students with limited programming experience can complete a credible engineering investigation.

Review of the previous outline

The original outline appropriately combined software engineering fundamentals, testing, architecture, collaboration and a semester project. These remain valuable. Its principal weakness for this cohort is the breadth of implementation technologies: several languages, multiple database families, Kubernetes, model deployment and multi-service integration were proposed within sixteen one-hour labs. That workload can reward copied configurations before students can explain the programs they run.

The second weakness is the emphasis on prompting and code generation without equally explicit treatment of agent permissions, repository context, probabilistic evaluation, data provenance and model-change regression. A program built with AI and a program containing AI are different engineering problems. Both now receive explicit treatment.

Original emphasis	Revision	Reason and location
Basic programming already assumed	Supported Python bridge and diagnostic	Weeks 1 to 4 establish readable code and independent testing
Prompt engineering concentrated in Week 7	Specification and context engineering across Weeks 5 to 7	Students manage requirements, files, evidence and tool scope
AI coding assistants	Assistants and bounded coding agents	Week 5 compares suggestion, editing and execution authority
Generic AI quality review	Code tests plus separate model evaluations	Weeks 4, 6 and 10 distinguish executable correctness from statistical performance
Multiple AI services in one lab	One replaceable AI adapter and a deterministic fallback	Week 10 makes failures, costs and evidence tractable
Broad architecture and infrastructure survey	Modular monolith, APIs, relational storage and one delivery pipeline	Weeks 8 to 12 prioritise a complete, understandable system
General ethics and security	Threat models, prompt injection, tool permissions and supply chain evidence	Introduced in Week 1, developed in Weeks 6 and 13
Performance and model optimisation	Profiling, tail latency, evaluation quality and cost per accepted result	Week 14 connects technical measurements to engineering decisions
Project demonstration	Reproducible release, comparative investigation and individual defence	Weeks 15 and 16 assess understanding and evidence

Evidence behind the update

The following observations support the revision. They are dated evidence, not predictions of a particular vendor's capabilities throughout 2027.

- DORA's 2025 report describes AI as amplifying the existing development system. The course therefore keeps review, small changes and reliable delivery alongside AI use. [R1]
- GitHub's current agent documentation describes tools that can modify repositories and run commands. Students must distinguish generated advice from delegated action and review the resulting changes. [R2]
- Anthropic's context engineering and agent evaluation articles describe explicit management of context and multi-step behaviour. These motivate dedicated learning activities on context selection and evaluation. [R3, R4]
- The MCP specification dated 28 July 2026 provides a current example of a protocol for connecting AI applications to tools and information. The course teaches the interface and trust boundary rather than requiring memorisation of a changing SDK. [R5]
- OWASP's official source repository identifies a published 2026 LLM Top 10. Security teaching uses that edition explicitly rather than relabelling the 2025 list. [R6]
- METR's February 2026 update explains why selection and measurement problems weakened its later productivity estimates. Students will measure their own tasks and qualify conclusions rather than repeat a universal AI speedup or slowdown claim. [R7]

Entry preparation and student support

Students should be comfortable with algebra, units, basic descriptive statistics and interpreting engineering data. Prior software engineering experience is not required. A short, ungraded diagnostic asks students to trace a calculation, interpret a conditional, identify a unit error and explain an expected result. Its purpose is to direct support, not exclude students.

Before Week 1, provide an approved Python 3 installation, an editor and the supplied course examples. VS Code is a suitable main editor; a simpler editor may be used for the bridge. Use one tested Python environment across the class. Record the exact interpreter and package versions in each project. Select supported versions when the course is deployed; this outline intentionally does not depend on a latest-version label.

Recommend four to six hours of independent study each week, including programming practice, reading and project work. This is a study recommendation, not an additional contact-hour or credit allocation. Students needing support should use the programming bridge, complete the core exercises and attend consultation before attempting extensions.

Course learning outcomes

On successful completion, a student will be able to:

1. **LO1 Requirements and reasoning:** Translate an electrical or information engineering problem into a bounded software specification with units, assumptions, quality requirements and independently justified acceptance examples.
2. **LO2 Implementation and verification:** Implement and explain modular Python software, manage versions, and construct tests that challenge its contracts, boundaries and failure behaviour.
3. **LO3 AI-assisted development:** Direct an assistant or coding agent using relevant context, limited authority and a reviewable task; evaluate its changes and maintain an honest assistance record.
4. **LO4 Architecture and integration:** Defend an architecture that separates deterministic computation, persistence and AI services, with explicit interfaces, data provenance and failure policies.
5. **LO5 Empirical evaluation:** Design and interpret a reproducible comparison using appropriate baselines, independent evaluation cases, uncertainty, latency and cost measurements.
6. **LO6 Security and operations:** Apply threat modelling, least privilege, dependency review, delivery checks, monitoring and recovery to an AI-enabled engineering application.
7. **LO7 Research and communication:** Critically assess current technical evidence and communicate the design, limitations and individual contribution of a reproducible engineering project.

Master's-level achievement is demonstrated by justified choices, experimental design, critique and transfer to unfamiliar cases. The course does not equate academic depth with the number of frameworks used.

Weekly schedule

Each ordinary week has a two-hour lecture and a one-hour lab. Detailed notes specify core work and optional extension work. Complete installation and large downloads before the scheduled lab.

Week	Module and lecture focus	One-hour lab and evidence
1	M1 Software engineering, requirements, AI roles and Python execution	Trace and run an energy calculation; write an acceptance example
2	M1 Functions, conditions, loops, collections and engineering contracts	Implement and explain a validated energy function
3	M2 Modular design, files, exceptions, Git and review	Separate computation from input and create a small reviewed change
4	M2 Test design, numerical checks and debugging	Test an energy function and critique a faulty implementation; proposal due
5	M3 Assistants, agents and specification-led development	Delegate one bounded change and audit the diff
6	M3 AI-generated code review and evaluation design	Use hidden acceptance cases and review an adversarial candidate
7	M3 Context engineering and reproducible task experiments	Compare two context packages on matched tasks
8	M4 Architecture, interfaces, quality attributes and design decisions	Produce a component design and one architecture decision record; milestone 1 due
9	M4 Data contracts, SQL, time series and retrieval foundations	Query a synthetic sensor dataset and examine retrieval failures
10	M4 AI adapters, structured outputs, RAG and bounded tool use	Exercise a local adapter and failure cases; demonstrate one approved live integration if available
11	M5 Agile work, collaboration, CI and dependency evidence	Execute a repeatable quality gate and review a small release candidate
12	M5 Deployment, observability, incidents and recovery	Run a local release and rehearse rollback; milestone 2 due
13	M6 AI application security, ethics and governance	Threat-model and test the simulated advice workflow
14	M6 Performance, reliability, cost and empirical comparison	Measure a bottleneck and justify a quality-cost decision
15	M7 Capstone integration, evaluation and technical reporting	Reproduce the final release and submit the project
16	M7 Synthesis, research critique and independent defence	Two-hour final examination; one-hour presentation and defence session

Module boundaries and teaching depth

Module 1 Foundations and the Python bridge, Weeks 1 to 2. Software lifecycle; requirements; engineering assumptions; deterministic versus AI behaviour; Python scripts, values, control flow and functions; algorithm tracing and simple complexity. Core examples use constant power and interval energy. Extension: invariants and error budgets.

Module 2 Reliable Python software, Weeks 3 to 4. Modules, data files, interfaces, exceptions, Git, review, unit and integration tests, numerical tolerances, regression tests and test independence. Extension: property-based and mutation testing. Testing starts before any autonomous implementation task.

Module 3 AI-assisted development and evaluation, Weeks 5 to 7. Task specifications, assistant and agent capabilities, context selection, repository guidance, bounded execution, review, maintainability, evaluation leakage and small comparative experiments. Extension: multi-agent coordination as a design problem. Students are not required to run multiple agents.

Module 4 Architecture and AI integration, Weeks 8 to 10. Modular monoliths, APIs, relational data, telemetry provenance, deterministic baselines, retrieval, structured response validation, tool interfaces and versioned evaluation. Extension: MCP implementation, embedding retrieval, asynchronous services or an additional AI provider.

Module 5 Collaborative delivery and operation, Weeks 11 to 12. Incremental work, CI, reproducible environments, dependency and build provenance, local deployment, observability, service objectives and recovery. Containers are a supported demonstration or extension. Kubernetes and infrastructure as code are comparative reading rather than mandatory practical deliverables.

Module 6 Security and empirical performance, Weeks 13 to 14. Trust boundaries, prompt injection, excessive agency, output handling, data exposure, supply chain risks, governance, profiling, tail latency, caching, cost and statistical interpretation. Quantisation and edge inference are design discussions; training or optimising a large model is not required.

Module 7 Capstone and synthesis, Weeks 15 to 16. Release evidence, reproducibility, comparative findings, technical writing, oral defence, examination preparation and critique of emerging claims. The project begins in Week 4 and accumulates evidence throughout the semester.

Teaching and learning approach

Start each new concept with an engineering question and a small worked example. Students predict outputs before execution, then explain discrepancies. Lecture time alternates short explanations, code reading and discussion of trade-offs. Labs use prepared environments and one narrow deliverable; substantial extensions belong to independent study.

Use a common running case: a simulated equipment energy and condition monitoring service. A deterministic Python component validates telemetry and computes energy or a threshold alarm. An optional AI component explains approved evidence from a maintenance manual. The model must not operate physical equipment. Other projects may use communications, signal-processing metadata or instrumentation data with equivalent scope.

Assessment and project requirements

The original weighting is retained: coursework 30% and final examination 70%. Coursework consists of the main project, 20%, and documentation and reflection, 10%. Practice questions, diagnostic work and routine lab completion are formative; they do not introduce extra percentage allocations.

Component	Percentage of course	Evidence
Main project requirements	4	Bounded problem, assumptions, traceable acceptance criteria
Main project architecture	3	Interfaces, data model and justified design decisions
Main project implementation	5	Working, understandable Python and one controlled AI feature or approved integration experiment
Main project verification	5	Independent tests, evaluation cases, failure checks and justified interpretation
Main project delivery	3	Repeatable execution, version record and recovery demonstration
Technical documentation	4	Architecture, usage, data and test documentation
AI assistance accountability	2	Accurate task and review records with individual explanation
Research evaluation and reflection	3	Baseline comparison, limitations and critical reading
Individual defence	1	Explain an unfamiliar change and defend evidence
Final examination	70	Independent reasoning across all seven modules
Total	100	Coursework 30 plus examination 70

Use teams of two or three where appropriate, with identifiable contributions and individual questioning. An equivalent individual project is acceptable. Assess understanding and evidence, not subscription tier, token expenditure or volume of generated code. The individual defence belongs within documentation and reflection; it is not an additional assessment category.

Week 4 proposal: one to two pages specifying the problem, users, permitted data, deterministic baseline, candidate AI contribution, three acceptance criteria and initial risks. Submit a small runnable Python function and initial tests.

Week 8 milestone 1: architecture, repository structure, baseline implementation, tests, data contract and at least one recorded design decision. Explain the AI boundary and identify the evaluation questions before tuning prompts.

Week 12 milestone 2: end-to-end prototype, replaceable adapter, failure handling, provisional evaluation suite, repeatable quality gate and a local deployment or equivalent managed environment. Keep final evaluation cases separate from development cases.

Week 15 submission: source release, instructions, dependency record, permitted dataset or generator, automated tests, evaluation inputs and raw results, technical report, AI assistance record and individual

contribution statement. Record actual results only. A live service is not necessary for marking if a replayable local demonstration and authentic integration evidence are supplied.

Week 16 defence: demonstrate the submitted release, explain a failed case and reason about a proposed modification. For a large class, use parallel examiner panels within the allocated session, supported by pre-submitted demonstrations. Arrange the panel plan before the semester; do not silently assume every group can give a long talk in one hour.

Final examination blueprint

Use a two-hour paper marked out of 100 and scaled to 70% of the course. Questions should require short code tracing, test design, architecture critique, security reasoning and interpretation of supplied empirical results. The proposed examination is independent and does not permit generative AI. Publish permitted reference materials and calculator arrangements before the examination under departmental rules.

Coverage	Marks out of 100	Principal outcomes
M1 Specification and Python reasoning	10	LO1 and LO2
M2 Verification and debugging	20	LO2
M3 AI workflow and evidence	20	LO3 and LO5
M4 Architecture and integration	20	LO4
M5 Delivery and operations	10	LO6
M6 Security and performance	15	LO5 and LO6
M7 Synthesis and research critique	5	LO7

LO7 is also assessed throughout written explanations and project evidence. Questions may integrate modules, but count their marks once in the blueprint.

AI use and academic integrity

Label activities with three clear assistance modes. **Independent** means no generative AI for the submitted attempt. **Guided** permits explanation or critique after a student's initial attempt within the stated task boundary. **Bounded development** permits generation and agent actions for a specified implementation task, subject to review and disclosure. These definitions are adopted for this course; they are not inherited restrictions from the reference course.

Students must be able to explain submitted code and independently check its important behaviour. Record tool and model identifier when exposed, date, task, relevant context, accepted and rejected changes, and verification evidence. Do not fabricate interactions or request private model reasoning; a concise observable action and decision record is sufficient. Declare reused code and sources. Never put access keys, private records or unapproved institutional data into prompts or repositories.

The project permits bounded AI use. Core concept checks and the final examination require independent work. The instructor may ask for a short modification or explanation to establish understanding. Follow

institutional academic integrity procedures for suspected misconduct; tool detectors alone are not reliable evidence of authorship.

Tools and access

Core tools are Python 3, an editor, Git, standard-library CSV and SQLite, and a testing tool such as pytest. Supplied examples can run with the Python standard library. One institutionally approved assistant or coding agent is enough; commercial tools and paid APIs may be used. The department should provide equitable access to the selected service or recorded integration results for students affected by outages or account limitations.

For AI integration, use one approved provider behind a replaceable adapter. Freeze an SDK version for each lab and consult its official documentation when preparing the live demonstration. Teach token accounting, budgets, data handling and model version changes without fixing an assumed price in the syllabus. No purchase or cloud deployment is necessary to read or execute the supplied local examples.

SQLite is the teaching database; PostgreSQL, embedding stores, containers and managed cloud services are extensions chosen to answer a project requirement. Multi-cloud deployment, a Kubernetes cluster and large-model training are outside the mandatory scope.

Reading and source register

Sources checked on 15 September 2026. The research baseline is current to that date; recheck product documentation and emerging guidance before each semester. No claims about later 2027 releases are assumed.

- [R1] DORA. State of AI-assisted Software Development 2025.
<https://dora.dev/research/2025/dora-report/>
- [R2] GitHub. Application card for GitHub Copilot Agents.
<https://docs.github.com/en/copilot/responsible-use/agents>
- [R3] Anthropic. Effective context engineering for AI agents.
<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- [R4] Anthropic. Demystifying evals for AI agents, 9 January 2026.
<https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- [R5] Model Context Protocol. Specification dated 28 July 2026.
<https://modelcontextprotocol.io/specification/2026-07-28>
- [R6] OWASP GenAI Security Project. Official source repository for the 2026 LLM Top 10.
<https://github.com/GenAI-Security-Project/GenAI-LLM-Top10>
- [R7] METR. We are Changing our Developer Productivity Experiment Design, 24 February 2026.
<https://metr.org/blog/2026-02-24-uplift-update/>
- [R8] Python Software Foundation. Python tutorial. <https://docs.python.org/3/tutorial/>
- [R9] pytest. Assertions and expected exceptions.
<https://docs.pytest.org/en/stable/how-to/assert.html>
- [R10] NIST. Secure Software Development Framework Version 1.1, SP 800-218.
<https://doi.org/10.6028/NIST.SP.800-218>
- [R11] NIST. AI Risk Management Framework and Generative AI Profile.
<https://www.nist.gov/itl/ai-risk-management-framework>

- [R12] OpenTelemetry. Signals. <https://opentelemetry.io/docs/concepts/signals/>
- [R13] SLSA. Specification Version 1.2. <https://slsa.dev/spec/v1.2/>

Core book reading: Ian Sommerville, Software Engineering; Titus Winters, Tom Manshreck and Hyrum Wright, Software Engineering at Google; Martin Kleppmann, Designing Data-Intensive Applications; David Thomas and Andrew Hunt, The Pragmatic Programmer. Use selected chapters on requirements, testing, collaboration, reliability and trade-offs rather than assigning whole books. The books establish durable concepts; dated primary sources support the AI updates.

Course pack structure

Each of the seven modules contains an overview, learning outcomes, a reading route, connected explanatory notes, engineering examples, common mistakes, timed weekly labs, twelve practice questions with model answers, revision prompts and references. Module 1 includes the programming bridge. Later modules retain accessible code while increasing the analytical depth. Core and extension work are explicitly distinguished.

HTML notes provide navigation, answer controls and offline reading. PDFs include the full notes and model answers for printing. Editable Markdown sources and runnable Python examples accompany the notes. The examples use invented classroom data; measured execution checks of supplied code do not imply that any external AI provider or cloud service has been tested.